STOCHASTIC ACTIVATION PRUNING FOR ROBUST ADVERSARIAL DEFENSE

Guneet S. Dhillon[1,2], **Kamyar Azizzadenesheli**[3], **Zachary C. Lipton**[1,4], **Jeremy Bernstein**[1,5], **Jean Kossaifi**[1,6], **Aran Khanna**[1], **Anima Anandkumar**[1,5]
[1]Amazon AI, [2]UT Austin, [3]UC Irvine, [4]CMU, [5]Caltech, [6]Imperial College London
guneetdhillon@utexas.edu, kazizzad@uci.edu, zlipton@cmu.edu,
bernstein@caltech.edu, jean.kossaifi@imperial.ac.uk,
aran@arankhanna.com, anima@amazon.com

arXiv:1803.01442v1 [cs.LG] 5 Mar 2018

ABSTRACT

Neural networks are known to be vulnerable to adversarial examples. Carefully chosen perturbations to real images, while imperceptible to humans, induce misclassification and threaten the reliability of deep learning systems in the wild. To guard against adversarial examples, we take inspiration from game theory and cast the problem as a minimax zero-sum game between the adversary and the model. In general, for such games, the optimal strategy for both players requires a stochastic policy, also known as a *mixed strategy*. In this light, we propose *Stochastic Activation Pruning* (SAP), a mixed strategy for adversarial defense. SAP prunes a random subset of activations (preferentially pruning those with smaller magnitude) and scales up the survivors to compensate. We can apply SAP to pretrained networks, including adversarially trained models, without fine-tuning, providing robustness against adversarial examples. Experiments demonstrate that SAP confers robustness against attacks, increasing accuracy and preserving calibration.

1 INTRODUCTION

While deep neural networks have emerged as dominant tools for supervised learning problems, they remain vulnerable to adversarial examples (Szegedy et al., 2013). Small, carefully chosen perturbations to input data can induce misclassification with high probability. In the image domain, even perturbations so small as to be imperceptible to humans can fool powerful convolutional neural networks (Szegedy et al., 2013; Goodfellow et al., 2014). This fragility presents an obstacle to using machine learning in the wild. For example, a vision system vulnerable to adversarial examples might be fundamentally unsuitable for a computer security application. Even if a vision system is not explicitly used for security, these weaknesses might be critical. Moreover, these problems seem unnecessary. If these perturbations are not perceptible to people, why should they fool a machine?

Since this problem was first identified, a rapid succession of papers have proposed various techniques both for *generating* and for *guarding against* adversarial attacks. Goodfellow et al. (2014) introduced a simple method for quickly producing adversarial examples called *the fast gradient sign method* (FGSM). To produce an adversarial example using FGSM, we update the inputs by taking one step in the direction of the *sign* of the gradient of the loss with respect to the input.

To defend against adversarial examples some papers propose training the neural network on adversarial examples themselves, either using the same model (Goodfellow et al., 2014; Madry et al., 2017), or using an ensemble of models (Tramèr et al., 2017a). Taking a different approach, Nayebi & Ganguli (2017) draws inspiration from biological systems. They propose that to harden neural networks against adversarial examples, one should learn flat, compressed representations that are sensitive to a minimal number of input dimensions.

This paper introduces *Stochastic Activation Pruning* (SAP), a method for guarding pretrained networks against adversarial examples. During the forward pass, we stochastically prune a subset of the activations in each layer, preferentially retaining activations with larger magnitudes. Following the pruning, we scale up the surviving activations to normalize the dynamic range of the inputs to the

subsequent layer. Unlike other adversarial defense methods, our method can be applied post-hoc to pretrained networks and requires no additional fine-tuning.

2 PRELIMINARIES

We denote an n-layered neural network $h : \mathcal{X} \to Y$ as a chain of functions $h = h^n \circ h^{n-1} \circ \ldots \circ h^1$, where each h^i consists of a linear transformation W^i followed by a non-linearity ϕ^i. Given a set of nonlinearities and weight matrices, a neural network provides a nonlinear mapping from inputs $x \in \mathcal{X}$ to outputs $\hat{y} \in \mathcal{Y}$, i.e.

$$\hat{y} := h(x) = \phi^n(W^n\phi^{n-1}(W^{n-1}\phi^{n-2}(\ldots\phi^1(W^1x)))).$$

In supervised classification and regression problems, we are given a data set $\mathcal{D}$ of pairs (x, y), where each pair is drawn from an unknown joint distribution. For the classification problems, y is a categorical random variable, and for regression, y is a real-valued vector. Conditioned on a given dataset, network architecture, and a loss function such as cross entropy, a learning algorithm, e.g. stochastic gradient descent, learns parameters $\theta := \{W^i\}_{i=1}^n$ in order to minimize the loss. We denote $J(\theta, x, y)$ as the loss of a learned network, parameterized by θ, on a pair of (x, y). To simplify notation, we focus on classification problems, although our methods are broadly applicable.

Consider an input x that is correctly classified by the model h. An adversary seeks to apply a small additive perturbation, Δx, such that $h(x) \neq h(x + \Delta x)$, subject to the constraint that the perturbation is imperceptible to a human. For perturbations applied to images, the l_∞-norm is considered a better measure of human perceptibility than the more familiar l_2 norm Goodfellow et al. (2014). Throughout this paper, we assume that the manipulative power of the adversary, the perturbation Δx, is of bounded norm $\|\Delta x\|_\infty \leq \lambda$. Given a classifier, one common way to generate an adversarial example is to perturb the input in the direction that increases the cross-entropy loss. This is equivalent to minimizing the probability assigned to the true label. Given the neural network h, network parameters θ, input data x, and corresponding true output y, an adversary could create a perturbation Δx as follows

$$\Delta x = \arg\max_{\|r\|_\infty \leq \lambda} J(\theta, x + r, y), \tag{1}$$

Due to nonlinearities in the underlying neural network, and therefore of the objective function J, the optimization Eq. 1, in general, can be a non-convex problem. Following Madry et al. (2017); Goodfellow et al. (2014), we use the first order approximation of the loss function

$$\Delta x = \arg\max_{\|r\|_\infty \leq \lambda} [J(\theta, x, y) + r^\top \mathcal{J}(\theta, x, y)], \qquad \text{where } \mathcal{J} = \nabla_x J.$$

The first term in the optimization is not a function of the adversary perturbation, therefore reduces to

$$\Delta x = \arg\max_{\|r\|_\infty \leq \lambda} r^\top \mathcal{J}(\theta, x, y).$$

An adversary chooses r to be in the direction of sign of $\mathcal{J}(\theta, x, y)$, i.e. $\Delta x = \lambda \cdot \text{sign}(\mathcal{J}(\theta, x, y))$. This is the FGSM technique due to Goodfellow et al. (2014). Note that FGSM requires an adversary to access the model in order to compute the gradient.

3 STOCHASTIC ACTIVATION PRUNING

Consider the defense problem from a game-theoretic perspective (Osborne & Rubinstein, 1994). The adversary designs a policy in order to maximize the defender's loss, while knowing the defenders policy. At the same time defender aims to come up with a strategy to minimize the maximized loss. Therefore, we can rewrite Eq. 1 as follows

$$\pi^*\ ,\ \rho^* := \arg\min_{\pi}\max_{\rho} \mathbb{E}_{p\sim\pi, r\sim\rho}\left[J(M_p(\theta), x + r, y)\right], \tag{2}$$

where ρ is the adversary's policy, which provides $r \sim \rho$ in the space of bounded (allowed) perturbations (for any r in range of ρ, $\|r\|_\infty \leq \lambda$) and π is the defenders policy which provides $p \sim \pi$, an instantiation of its policy. The adversary's goal is to maximize the loss of the defender by perturbing

Algorithm 1 Stochastic Activation Pruning (SAP)

Input: input datum x, neural network with n layers, with i^{th} layer having weight matrix W^i, non-linearity ϕ^i and number of samples to be drawn r^i.
$h^0 \leftarrow x$
for each layer i **do**
  $h^i \leftarrow \phi^i(W^i h^{i-1})$ ▷ activation vector for layer i with dimension a^i
  $p^i_j \leftarrow \frac{|(h^i)_j|}{\sum_{k=1}^{a^i} |(h^i)_k|},\ \forall j \in \{1, \ldots, a^i\}$ ▷ activations normalized on to the simplex
  $S \leftarrow \{\}$ ▷ set of indices not to be pruned
  repeat r^i **times** ▷ the activations have r^i chances of being kept
    Draw $s \sim \text{categorical}(p^i)$ ▷ draw an index to be kept
    $S \leftarrow S \cup \{s\}$ ▷ add index s to the keep set
  for each $j \notin S$ **do**
    $(h^i)_j \leftarrow 0$ ▷ prune the activations not in S
  for each $j \in S$ **do**
    $(h^i)_j \leftarrow \frac{(h^i)_j}{1-(1-p^i_j)^{r^i}}$ ▷ scale up the activations in S
return h^n

the input under a strategy ρ and the defender's goal is to minimize the loss by changing model parameters θ to $M_p(\theta)$ under strategy π. The optimization problem in Eq. 2 is a *minimax* zero-sum game between the adversary and defender where the optimal strategies (π^*, ρ^*), in general, are mixed Nash equilibrium, i.e. stochastic policies.

Intuitively, the idea of SAP is to stochastically drop out nodes in each layer during forward propagation. We retain nodes with probabilities proportional to the magnitude of their activation and scale up the surviving nodes to preserve the dynamic range of the activations in each layer. Empirically, the approach preserves the accuracy of the original model. Notably, the method can be applied post-hoc to already-trained models.

Formally, assume a given pretrained model, with activation layers (***ReLU, Sigmoid, etc.***) and input pair of (x, y). For each of those layers, SAP converts the activation map to a multinomial distribution, choosing each activation with a probability proportional to its absolute value. In other words, we obtain the multinomial distribution of each activation layer with L_1 normalization of their absolute values onto a L_1-ball simplex. Given the i'th layer activation map, $h^i \in \mathbb{R}^{a^i}$, the probability of sampling the j'th activation with value $(h^i)_j$ is given by

$$p^i_j = \frac{|(h^i)_j|}{\sum_{k=1}^{a^i} |(h^i)_k|}.$$

We draw random samples with replacement from the activation map given the probability distribution described above. This makes it convenient to determine whether an activation would be sampled at all. If an activation is sampled, we scale it up by the inverse of the probability of sampling it over all the draws. If not, we set the activation to 0. In this way, SAP preserves inverse propensity scoring of each activation. Under an instance p of policy π, we draw r^i_p samples with replacement from this multinomial distribution. The new activation map, $M_p(h^i)$ is given by

$$M_p(h^i) = h^i \odot m^i_p, \qquad (m^i_p)_j = \frac{\mathbb{I}((h^i)_j)}{1-(1-p^i_j)^{r^i_p}},$$

where $\mathbb{I}((h^i)_j)$ is the indicator function that returns 1 if $(h^i)_j$ was sampled at least once, and 0 otherwise. The algorithm is described in Algorithm 1. In this way, the model parameters are changed from θ to $M_p(\theta)$, for instance p under policy π, while the reweighting $1-(1-p^i_j)^{r^i_p}$ preserves $\mathbb{E}_{p\sim\pi}[M_p(h^i)_j] = (h^i)_j$. If the model was linear, the proposed pruning method would behave the same way as the original model in expectation. In practice, we find that even with the non-linearities in deep neural networks, for sufficiently many examples, SAP performs similarly to the un-pruned model. This guides our decision to apply SAP to pretrained models without performing fine-tuning.